

CH1 – RAPPEL 2

Les fonctions

Sommaire :

- Avantages d'un programme modulaire
- Définition d'une fonction
- Appel à une fonction
- Les paramètres
- Passage de tableaux en paramètres
- Passage de paramètres par référence
- Paramètres en entrées et en sorties
- Passer un tableau de taille quelconque
- Portée des variables
- Conclusion
- Exercices corrigés

Pourquoi écrire des fonctions

Lorsqu'on a un ensemble de lignes de code qui doivent être exécutées à différents endroits dans un programme, au lieu de réécrire les mêmes lignes de code, il est intéressant de créer des fonctions (modules).

Au lieu d'écrire une fonction `main()` de 500 lignes, il est préférable de créer 25 fonctions de 20 lignes

- on structure le programme en modules.
- il est plus facile de tester chaque fonction (module).

Il est impossible d'avoir en tête plus de 1000 lignes de codes : or la plupart des programmes réels comportent des dizaines de milliers de lignes et les grosses applications en comportent des millions. Ecrire des fonctions est absolument obligatoire.

Voici quelques **avantages d'un programme modulaire**:

- **Meilleure lisibilité**
- **Diminution du risque d'erreurs**
- **Possibilité de tests sélectifs**
- **Dissimulation des méthodes**

Lors de l'utilisation d'un module il faut seulement connaître son effet, sans devoir s'occuper des détails de sa réalisation.

- **Réutilisation de modules déjà existants**

Il est facile d'utiliser des modules qu'on a écrits soi-même ou qui ont été développés par d'autres personnes.

- **Simplicité de l'entretien**

Un module peut être changé ou remplacé sans devoir toucher aux autres modules du programme.

- **Favorisation du travail en équipe**

Un programme peut être développé en équipe par délégation de la programmation des modules à différentes personnes ou groupes de personnes. Une fois développés, les modules peuvent constituer une base de travail commune.

- **Hiérarchisation des modules**

Un programme peut d'abord être résolu globalement au niveau du module principal. Les détails peuvent être reportés à des modules sous-ordonnés qui peuvent eux aussi être subdivisés en sous-modules et ainsi de suite. De cette façon, nous obtenons une *hiérarchie de modules*.

Les modules peuvent être développés en passant du haut vers le bas dans la hiérarchie (*'top-down-*

development' - méthode du raffinement progressif) ou bien en passant du bas vers le haut (*'bottom-up-development'*).

Dans la définition d'une fonction, nous indiquons:

- le nom de la fonction
- le type, le nombre et les noms des paramètres de la fonction
- le type du résultat fourni par la fonction
- les données locales à la fonction
- les instructions à exécuter

Définition d'une fonction en langage algorithmique

```
fonction <NomFonct> (<NomPar1>, <NomPar2>, ...):<TypeRés>  
| <déclarations des paramètres>  
| <déclarations locales>  
| <instructions>  
ffonction
```

Définition d'une fonction en C

```
<TypeRés> <NomFonct> (<TypePar1> <NomPar1>,  
                      <TypePar2> <NomPar2>, ... )  
  
{  
    // Corps de la fonction  
    <déclarations locales>  
    <instructions>  
}
```

Remarquez

Qu'il n'y a pas de point-virgule derrière la définition des paramètres de la fonction.

A chaque appel de la fonction on exécute le corps de la fonction.

L'identificateur est le nom de la fonction.

La fonction peut avoir des paramètres.

La fonction peut renvoyer une valeur de type type.

Appel à une fonction

Lors de l'appel de la fonction, le programme exécute la totalité des instructions du corps de la fonction, puis reprend le programme appelant juste après l'instruction d'appel de la fonction.

Exemple 1 : un exemple de fonction

```
#include <iostream>  
using namespace std;  
  
void b()  
{  
    cout<<"Bonjour"<<endl;  
}  
  
int main()  
{  
    cout<<"SALEM1"<<endl;  
    b();  
    cout<<" SALEM 2"<<endl;  
    b();  
    b();  
    cout<<" SALEM 3"<<endl;  
    b();  
    system("pause");  
    return 0;  
}
```

Explications

- Dans ce programme, on a créé une fonction b qui se contente d'afficher "Bonjour" à l'écran. La fonction b est précédée du type void : cela signifie que la fonction ne renvoie aucune valeur au programme appelant.
- Le programme principal (la fonction main()) affiche " SALEM 1" à l'écran, ensuite appelle la fonction b, affiche le message " SALEM 2" à l'écran, appelle ensuite 2 fois la fonction b, affiche le message " SALEM 3" et appelle une dernière fois la fonction b.

Exécution de l'exemple 1

Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran:

```
SALEM 1
Bonjour
SALEM 2
Bonjour
Bonjour
SALEM 3
Bonjour
```

Les paramètres

On peut paramétrer une fonction : les paramètres permettent de rendre la fonction plus générale donc plus facilement réutilisable. La réutilisation de code est une des notions fondamentales du langage C++. La notion de fonction est une des premières méthodes nous permettant d'aborder le sujet.

Exemple 2 : une fonction avec des paramètres

```
#include <iostream>
using namespace std;
void b(int i)
{
    int j;
    for(j=0; j<i; j++)
        cout<<"Bonjour"<<endl;
}
int main()
{
    cout<<" SALEM 1"<<endl;
    b(2);
    cout<<" SALEM 2"<<endl;
    b(3);
    system("pause");
    return 0;
}
```

Explications

- Cette fois-ci la fonction b comporte un paramètre entier i. Cette fonction affiche i fois "Bonjour" à l'écran.
- Dans le programme principal, la fonction main() affiche " SALEM 1", appelle ensuite la fonction b avec comme paramètre 2, **Lors de l'appel b(2) de la fonction, on recopie la valeur 2 dans i puis on exécute le corps de la fonction avec cette valeur de i : on affiche donc 2 fois "Bonjour".
- Lors de l'appel b(3) de la fonction, on recopie la valeur 3 dans i puis on exécute le corps de la fonction avec cette valeur de i : on affiche donc 3 fois "Bonjour".

Exécution

Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :

```
SALEM 1
Bonjour
Bonjour
SALEM 2
Bonjour
Bonjour
Bonjour
```

Les paramètres

On peut paramétrer une fonction : les paramètres permettent de rendre la fonction plus générale donc plus facilement réutilisable. La réutilisation de code est une des notions fondamentales du langage C++. La notion de fonction est une des premières méthodes nous permettant d'aborder le sujet.

Exemple 2 : une fonction avec des paramètres

```
#include <iostream>
using namespace std;

void b(int i)
{
    int j;
    for(j=0; j<i; j++)
        cout<<"Bonjour"<<endl;
}

int main()
{
    cout<<"SALEM1"<<endl;
    b(2);
    cout<<" SALEM 2"<<endl;
    b(3);
    system("pause");
    return 0;
}
```

• Explications

- Cette fois-ci la fonction b comporte un paramètre entier i. Cette fonction affiche i fois "Bonjour" à l'écran.
- Dans le programme principal, la fonction main() affiche " SALEM 1", appelle ensuite la fonction b avec comme paramètre 2, **Lors de l'appel b(2) de la fonction, on recopie la valeur 2 dans i puis on exécute le corps de la fonction avec cette valeur de i : on affiche donc 2 fois "Bonjour".
- Lors de l'appel b(3) de la fonction, on recopie la valeur 3 dans i puis on exécute le corps de la fonction avec cette valeur de i : on affiche donc 3 fois "Bonjour".

• Exécution

Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :

```
SALEM 1
Bonjour
Bonjour
SALEM 2
Bonjour
Bonjour
Bonjour
```

Environnement d'une fonction

- Une fonction peut avoir ses propres variables locales.
- On appelle environnement d'une fonction l'ensemble des variables accessibles dans le corps de la fonction.
- Il est possible de définir des variables globales (voir la définition des constantes) : **il est fortement déconseillé d'utiliser des variables globales autres que des constantes.**
- L'environnement d'une fonction comprend :
 - les variables locales à la fonction.
 - les paramètres de la fonction.
 - les variables globales.

Une fonction qui renvoie une valeur

Une fonction peut renvoyer une valeur d'un certain type à l'environnement appelant. **void signifie que la fonction ne renvoie rien.** Si on veut que la fonction renvoie une certaine valeur, il faudra écrire un autre type à la place de void. Le mot clé return permet :

- d'arrêter l'exécution de la fonction (il est en général placé à la fin de la fonction)

- de renvoyer une valeur à l'environnement appelant. Si une fonction `b` renvoie un entier, l'appel s'écrira : `a=b(...)`; `a` permettra de récupérer la valeur renvoyée par le `return`.

Exemple 3 : une fonction qui renvoie une valeur

```
#include<iostream>
using namespace std;
int b(int i, int j)
{ int k;
  k = i*i + j*j;
  return k;
}
int main()
{ int a;
  a = b(3,4);
  cout<<"Le resultat vaut : "<<a<<endl;
  return 0;
}
```

• Explications

Lors de l'appel `a=b(3,4)`;

- 3 et 4 sont respectivement copiés dans les variables `i` et `j`.
- On exécute le corps de la fonction (qui contient une variable locale `k`)
- `return k` permet d'arrêter l'exécution de la fonction en renvoyant la valeur de `k` qui est copiée dans `a`.
- L'environnement de la fonction `main` ne comprend que la variable `a` : on ne peut pas utiliser les variables `i`, `j` et `k` dans la fonction `main`.
- L'environnement de la fonction `b` comprend les variables `i`, `j` et `k` : on ne peut pas utiliser `a` dans la fonction `b`.
- La séparation des environnements permet de mieux structurer les applications.

• Exécution :

Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :

Le résultat vaut : 25

Exemple 4 : une autre fonction

```
#include <iostream>
using namespace std;
int b(int i)
{ int k, s=0;
  for(k=1; k<=i; k++)
    s = s+k*k;
  return s;
}

int main()
{ int a; a = b(4);
  cout<<"Le résultat vaut : "<<a<<endl;
  return 0;
}
```

• Explications

- On définit une fonction `b` qui a un paramètre entier `i` et qui calcule la somme des carrés des `i` premiers entiers.
- Dans la fonction `main`, on appelle `b(4)` et on récupère la valeur renvoyée dans la variable `a`. `a` contient donc la somme des 4 premiers carrés.

• Exécution

Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :

Le résultat vaut : 30

Exemple 5 : une fonction manipulant des réels

```
#include <iostream>
using namespace std;
double b(double x, double y)
{ double m; m = (x+y) / 2;
  return m;
}

int main()
{ double a; a = b(3.2, 4.2);
  cout<<"Le résultat vaut :"<<a<<endl;
  return 0;
}
```

• Explications :

- Dans cet exemple, la fonction b a 2 paramètres de type double, nommées x et y. La fonction b renvoie un double. Cette fonction renvoie la moyenne de x et de y. La variable m est une variable locale à la fonction b.
- Dans le programme principal, on récupère dans a la moyenne de 3.2 et 4.2 en appelant la fonction b.

• Exécution :

Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :
Le résultat vaut : 3.7

Compatibilité des types

Une fonction peut avoir des paramètres de différents types. Lors de l'appel, il convient de respecter l'ordre et le type des paramètres entre l'entête et le corps de la fonction.

Passage de tableaux en paramètres

Lorsqu'on passe un tableau (d'entiers par exemple en paramètre), il y a identification entre le tableau de l'environnement appelant et le paramètre de la fonction. Toute modification du tableau dans la fonction est répercutée dans le tableau de l'environnement appelant.

Exemple 6 : passage de tableaux en paramètre

```
#include <iostream>
using namespace std;
const int n=4;
void saisir(int t[n])
{ int i;
  for(i=0; i<n; i++)
  { cout<<"Tapez la valeur numero "<<i<<" : ";
    cin >> t[i];
  }
}

void affiche(int t[n])
{ int i;for(i=0; i<n; i++)
  cout<<"La valeur numero "<<i<<" est : "<<t[i]<<endl;
}

int main()
{ int a[n];
  saisir(a);
  affiche(a);
  return 0;
}
```

• Explications

- Lors de l'appel saisir(a), il y a identification du tableau a et du paramètre t de la fonction saisir : toute modification de t modifie le tableau a. La fonction saisir a permet de demander à l'utilisateur de saisir une à une toutes les cases d'un tableau de n cases.
- La fonction affiche un tableau de n entiers en paramètres et affiche toutes les cases de ce tableau.
- Le programme est structuré : il est constitué d'un ensemble de fonctions courtes dont le rôle peut être facilement identifié.
- Le main() devient un programme très court !

• Exécution

Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :

```
Tapez la valeur numero 0 : 4
Tapez la valeur numero 1 : 6
Tapez la valeur numero 2 : 5
Tapez la valeur numero 3 : 7
La valeur numero 0 est : 4
La valeur numero 1 est : 6
La valeur numero 2 est : 5
La valeur numero 3 est : 7
```

Attention

Une modification d'un paramètre qui n'est pas un tableau dans le corps d'une fonction n'est pas répercuté dans l'environnement appelant.

Exemple 7

```
#include <iostream>
using namespace std;

void saisir(int n)
{ cout<<"Tapez un entier : ";
  cin>>n;
}

void affiche(int n)
{ cout<<"La valeur de l'entier est : "<<n<<endl;
}

int main()
{ int x; x=54;
  saisir(x);
  affiche(x);
  return 0;
}
```

Explications

- Dans la fonction main, on définit une variable entière x et on l'initialise à 54.
- On appelle ensuite la fonction saisir : on copie la valeur de x dans n. Dans le corps de la fonction saisir, on demande à l'utilisateur de taper une valeur qui est mise dans x. Mais à aucun moment cette valeur x n'est mise dans la variable x : la valeur tapée par l'utilisateur est donc perdue.
- La valeur de x reste donc à 54. Lorsqu'on appelle la fonction affiche, on affiche la valeur 54.

Exécution

Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :

```
Tapez un entier : 20
La valeur de l'entier : 54
```

Passage de paramètres par référence

On peut passer un paramètre par référence (et non par copie) en indiquant dans l'entête de la fonction un & après le type. Il y a alors identification du paramètre de la fonction et de la variable de l'environnement appelant.

Exemple 8 : passage de paramètres par référence

```
#include <iostream>
using namespace std;

void saisir(int & n)
{ cout<<"Tapez un entier : ";
  cin>>n;
}

void affiche(int n)
{ cout<<"La valeur de l'entier : "<<n<<endl;
}

int main()
{ int x; x=54;
  saisir(x);
  affiche(x);
  return 0;
}
```

Explications

- Lors de l'appel saisir(x), il y a identification des variables x et n : toute modification de n modifie la valeur de x. Lorsque l'utilisateur saisit la valeur de n dans l'instruction cin>>n; , il y a modification du contenu de la variable x. On récupère dans x la valeur tapée par l'utilisateur
- On appelle ensuite la fonction affiche qui affiche la valeur de x.

Exécution

Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :

Tapez un entier : 20

La valeur de l'entier: 20

Paramètres en entrées et en sorties

- Certains paramètres vont fournir des données à une fonction : les paramètres en entrée.
- D'autres vont permettre de renvoyer une valeur vers l'environnement appelant : les paramètres en sortie.
- Techniquement, on utilisera le return ou le passage de paramètres par référence pour envoyer une valeur à l'environnement appelant.
- D'autres peuvent éventuellement avoir les 2 rôles : les paramètres en entrée/sortie.

Exemple 9 : paramètres en entrées et en sorties

```
#include <iostream>
using namespace std;

void minmax(int i, int j, int & min, int & max)
{if(i<j)
  {min=i; max=j;}
else {min=j; max=i;};
}

int main()
{ int a,b,w, x;
  cout << "Tapez la valeur de a : "; cin >> a;
  cout << "Tapez la valeur de b : "; cin >> b;
  minmax(a,b,w,x);
  cout << "Le plus petit vaut : " << w << endl;
  cout << "Le plus grand vaut : " << x << endl;
  return 0;
}
```

Explications

- La fonction minmax permet de récupérer le plus petit et le plus grand de 2 entiers i et j.
- Dans la fonction minmax, i et j sont les paramètres en entrée et min et max sont les paramètres en sorties.
- Dans le main, lorsqu'on appelle la fonction minmax, on recopie les valeurs de a et de b respectivement dans i et j et il y a identification des variables w et min et des variables x et max durant toute la durée de l'appel de la fonction.
- Dans w on récupère donc le plus petit de a et de b et dans x le plus grand de a et de b.

Exécution

Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :

Tapez la valeur de a : 15

La valeur de b est : 11

Le plus petit vaut 11

Le plus grand vaut 15

Passer un tableau de taille quelconque

- Le paramètre d'une fonction peut être un tableau de taille quelconque.
- Dans l'environnement appelant, la taille du tableau devra être déterminée.
- En général, lorsqu'on passe un tableau de taille quelconque, on passe un autre paramètre entier qui indique la taille du tableau.

Exemple 10 : passer un tableau de taille quelconque

```
#include <iostream>
using namespace std;
void saisir(int t[], int n)
{
    int i;
    for(i=0; i<n; i++)
    {
        cout << "Tapez l'entier numero " << i << " : ";
        cin >> t[i];
    }
}

void afficher(int t[], int n)
{
    int i;
    for(i=0; i<n; i++)
        cout << "L'entier numero " << i << " vaut : " << t[i] << endl;
}

int main()
{
    int a[3], b[5];
    cout << "SAISIE DU TABLEAU a" << endl;
    saisir(a, 3);
    cout << "SAISIE DU TABLEAU b" << endl;
    saisir(b, 5);
    cout << "AFFICHAGE DU TABLEAU a" << endl;
    afficher(a, 3);
    cout << "AFFICHAGE DU TABLEAU b" << endl;
    afficher(b, 5);
    return 0;
}
```

Explications

- Dans la fonction saisir, le premier paramètre est int t[] : il s'agit donc d'un tableau d'entiers de n'importe quelle taille. Le deuxième paramètre n indique la taille du tableau. Cette fonction demande à l'utilisateur de saisir une à une toutes les cases du tableau.
- La fonction affiche a également comme paramètres un tableau d'entiers de taille quelconque et un paramètre n qui indique la taille du tableau. Cette fonction affiche le contenu du tableau.

- Dans le programme principal, on saisit le contenu du tableau a contenant 3 cases et du tableau b contenant 5 cases en

**** On constate toutefois que dans la fonction main, la taille des tableaux a et b est belle et bien connue.**

Exécution

Lorsqu'on exécute le programme voici ce qu'on obtient à l'écran :

SAISIE DU TABLEAU a

Tapez l'entier numero 0: ""123""

Tapez l'entier numero 1: ""456""

Tapez l'entier numero 2: ""789""

SAISIE DU TABLEAU b

Tapez l'entier numero 0: ""987""

Tapez l'entier numero 1: ""654""

Tapez l'entier numero 2: ""321""

Tapez l'entier numero 3: ""741""

Tapez l'entier numero 4: ""852""

AFFICHAGE DU TABLEAU a

L'entier 0 vaut : 123

L'entier 1 vaut : 456

L'entier 2 vaut : 789

AFFICHAGE DU TABLEAU b

0 vaut : 987

L'entier 1 vaut : 654

L'entier 2 vaut : 321

L'entier 3 vaut : 741

L'entier 4 vaut : 852

Portée des variables

- Dans la main, on peut accéder :
 - aux variables locales du main
 - aux constantes globales
- Dans une fonction, on peut accéder
 - aux paramètres de la fonction
 - aux constantes globales
- Il est déconseillé de définir des variables globales non constantes !
- Les échanges entre les environnements se font :
 - par les returns
 - par les passages de paramètres par référence

Conclusion

- Il faudra maîtriser les passages de paramètres par valeur et par référence ainsi que le rôle particulier des tableaux.
- Il est indispensable d'avoir bien compris le rôle des paramètres en entrée et en sortie d'une fonction.
- Les fonctions vont jouer un rôle fondamental dans la structuration de nos programmes. Nos programmes vont maintenant être constitués d'une multitude de fonctions qui auront une taille petite (10 à 30 lignes) et qui auront toutes un rôle très précis.
- Cette méthodologie est indispensable pour créer de longs programmes.

Exercices

NB : Adapter les solutions proposées à votre environnement de développement

Exercice 1

Ecrire un programme se servant d'une fonction F pour afficher la table de valeurs de la fonction définie par

$$f(x) = \sin(x) + \ln(x) - \sqrt{x}$$

où x est un entier compris entre 1 et 10.

Solution

```
#include <stdio.h>
#include <math.h>
main()
{
    /* Prototypes des fonctions appelées */
    double F(int X);
    /* Variables locales */
    int I;
    /* Traitements */
    printf("\tX\tF(X)\n");
    for (I=1 ; I<=10 ; I++)
        printf("\t%d\t%f\n", I, F(I));
    return 0;
}

double F(int X)
{
    return sin(X)+log(X)-sqrt(X);
}
```

Exercice 2

Ecrire la fonction NCHIFFRES du type **int** qui obtient une valeur entière N (positive ou négative) du type **long** comme paramètre et qui fournit le nombre de chiffres de N comme résultat.

Ecrire un petit programme qui teste la fonction NCHIFFRES:

Exemple:

```
Introduire un nombre entier : 6457392
Le nombre 6457392 a 7 chiffres.
```

Solution 2

```
#include <stdio.h>
main()
{
    /* Prototypes des fonctions appelées */
    int NCHIFFRES(long N);
    /* Variables locales */
    long A;
    /* Traitements */
    printf("Introduire un nombre entier : ");
    scanf("%ld", &A);
    printf("Le nombre %ld a %d chiffres.\n", A, NCHIFFRES(A));
    return 0;
}

int NCHIFFRES(long N)
{
    /* Comme N est transmis par valeur, N peut être */
    /* modifié à l'intérieur de la fonction. */
```

```

int I;
/* Conversion du signe si N est négatif */
if (N<0)
    N *= -1;
/* Compter les chiffres */
for (I=1; N>10; I++)
    N /= 10;
return I;
}

```

Exercice 3 « Tri par propagation »

Ecrire la fonction TRI_BULLE qui trie un tableau de N éléments entiers par ordre croissant en appliquant la méthode de la bulle (tri par propagation - voir exercice 7.15). Employer la fonction RANGER de l'exercice ci-dessus.

Ecrire un programme pour tester la fonction TRI_BULLE.

Solution 3 « Tri par propagation »

```

#include <stdio.h>
main()
{
    /* Prototypes des fonctions appelées */
    void LIRE_TAB (int *TAB, int *N, int NMAX);
    void TRI_BULLE(int *T, int N);
    void ECRIRE_TAB (int *TAB, int N);
    /* Variables locales */
    int T[100]; /* Tableau d'entiers */
    int DIM;    /* Dimension du tableau */
    /* Traitements */
    LIRE_TAB (T, &DIM, 100);
    printf("Tableau donné : \n");
    ECRIRE_TAB (T, DIM);
    TRI_BULLE(T, DIM);
    printf("Tableau trié : \n");
    ECRIRE_TAB (T, DIM);
    return 0;
}

void TRI_BULLE(int *T, int N)
{
    /* Prototypes des fonctions appelées */
    int RANGER(int *X, int *Y);
    /* Variables locales */
    int I,J; /* indices courants */
    int FIN; /* position où la dernière permutation a eu lieu */
    /* permet de ne pas trier un sous-ensemble déjà trié. */
    /* Tri de T par propagation de l'élément maximal */
    for (I=N-1 ; I>0 ; I=FIN)
    {
        FIN=0;
        for (J=0; J<I; J++)
            if (RANGER(T+J, T+J+1)) FIN = J;
    }
}

int RANGER(int *X, int *Y)
{
    . . .
}

void LIRE_TAB (int *TAB, int *N, int NMAX)
{
    . . .
}

```

```
void ECRIRE_TAB (int *TAB, int N)
{
    . . .
}
```

Exercice 4

Ecrire la fonction ADDITION_MATRICE qui effectue l'addition des matrices suivante:

$$\text{MAT1} = \text{MAT1} + \text{MAT2}$$

Choisir les paramètres nécessaires et écrire un petit programme qui teste la fonction ADDITION_MATRICE.

Solution 4

```
#include <stdio.h>

main()
{
    /* Prototypes des fonctions appelées */
    void ADDITION_MATRICE (int *MAT1, int *MAT2, int L, int C, int CMAX);
    void LIRE_DIM (int *L, int LMAX, int *C, int CMAX);
    void LIRE_MATRICE (int *MAT, int L, int C, int CMAX);
    void ECRIRE_MATRICE (int *MAT, int L, int C, int CMAX);
    /* Variables locales */
    /* Les matrices et leurs dimensions */
    int M1[30][30], M2[30][30];
    int L, C;
    /* Traitements */
    LIRE_DIM (&L, 30, &C, 30);
    printf("*** Matrice 1 ***\n");
    LIRE_MATRICE ((int*)M1, L, C, 30);
    printf("*** Matrice 2 ***\n");
    LIRE_MATRICE ((int*)M2, L, C, 30);
    printf("Matrice donnée 1 : \n");
    ECRIRE_MATRICE ((int*)M1, L, C, 30);
    printf("Matrice donnée 2 : \n");
    ECRIRE_MATRICE ((int*)M2, L, C, 30);
    ADDITION_MATRICE( (int*)M1, (int*)M2, L, C, 30);
    printf("Matrice résultat : \n");
    ECRIRE_MATRICE ((int*)M1, L, C, 30);
    return 0;
}

void ADDITION_MATRICE (int *MAT1, int *MAT2, int L, int C, int CMAX)
{
    /* Variables locales */
    int I, J;
    /* Ajouter les éléments de MAT2 à MAT1 */
    for (I=0; I<L; I++)
        for (J=0; J<C; J++)
            *(MAT1+I*CMAX+J) += *(MAT2+I*CMAX+J);
}

void LIRE_DIM (int *L, int LMAX, int *C, int CMAX)
{
    . . .
}

void LIRE_MATRICE (int *MAT, int L, int C, int CMAX)
{
    . . .
}
```

```
void ECRIRE_MATRICE (int *MAT, int L, int C, int CMAX)
{
    . . .
}
```

Exercice 5

Ecrire la fonction MULTI_MATRICE qui effectue la multiplication de la matrice MAT1 par un entier X:

$$\text{MAT1} = \text{X} * \text{MAT1}$$

Choisir les paramètres nécessaires et écrire un petit programme qui teste la fonction MULTI_MATRICE.

Exercice 5

```
#include <stdio.h>

main()
{
    /* Prototypes des fonctions appelées */
    void MULTI_MATRICE(int X, int *MAT, int L, int C, int CMAX);
    void LIRE_DIM      (int *L, int LMAX, int *C, int CMAX);
    void LIRE_MATRICE  (int *MAT, int L, int C, int CMAX);
    void ECRIRE_MATRICE (int *MAT, int L, int C, int CMAX);
    /* Variables locales */
    int M[30][30]; /* Matrice d'entiers */
    int L, C;      /* Dimensions de la matrice */
    int X;
    /* Traitements */
    LIRE_DIM (&L,30,&C,30);
    LIRE_MATRICE ((int*)M,L,C,30 );
    printf("Introduire le multiplicateur (entier) : ");
    scanf("%d", &X);
    printf("Matrice donnée : \n");
    ECRIRE_MATRICE ((int*)M,L,C,30);
    MULTI_MATRICE (X,(int*)M,L,C,30);
    printf("Matrice résultat : \n");
    ECRIRE_MATRICE ((int*)M,L,C,30);
    return 0;
}

void MULTI_MATRICE(int X, int *MAT, int L, int C, int CMAX)
{
    /* Variables locales */
    int I,J;
    /* Multiplication des éléments */
    for (I=0; I<L; I++)
        for (J=0; J<C; J++)
            *(MAT+I*CMAX+J) *= X;
}

void LIRE_DIM (int *L, int LMAX, int *C, int CMAX)
{
    . . .
}

void LIRE_MATRICE (int *MAT, int L, int C, int CMAX)
{
    . . .
}

void ECRIRE_MATRICE (int *MAT, int L, int C, int CMAX)
{
    . . .
}
```